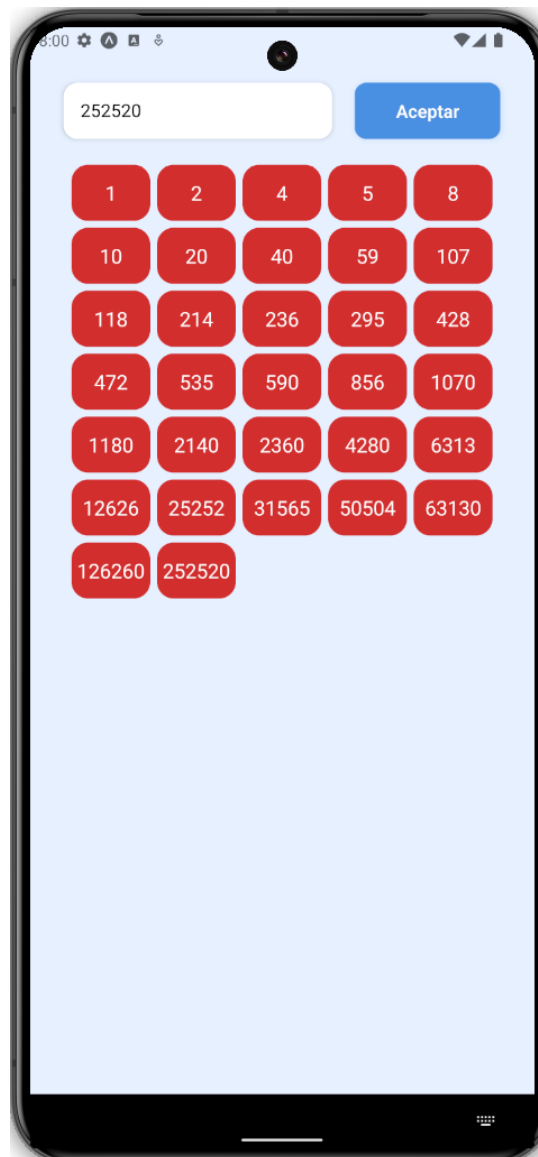


# TUTORIAL 11

## FLATLIST

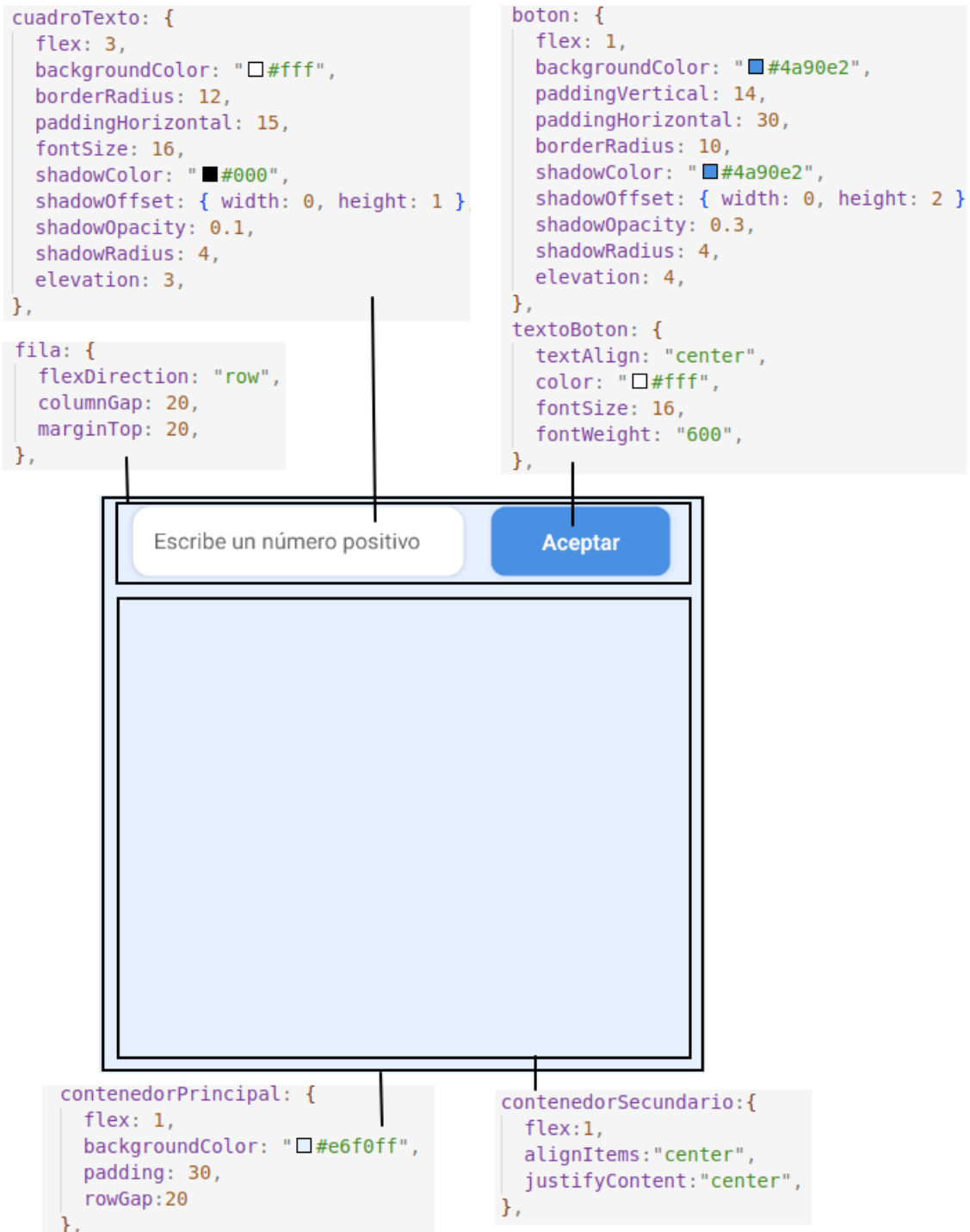


Casi todas las apps hacen uso de listas o tablas para mostrar datos que se obtienen dinámicamente. Por ejemplo, un usuario puede introducir un número y podemos mostrar una lista con todos sus divisores. Aunque la generación dinámica de componentes es una técnica que podría usarse para conseguir esto, React Native cuenta con un componente mucho más eficiente para ello: **FlatList**

## 1.- Diseño de la interfaz

Vamos a realizar una app que encuentre los divisores de un número y los muestre en un cuadro de texto. Comenzaremos diseñando la interfaz

1. Crea un proyecto local llamado **rn\_divisores** y súbelo a GitHub
2. Crea una rama llamada **tutorial11**
3. Abre **App.tsx** y borra todo lo que hay
4. Teclea **rnfs** y pulsa **Intro**
5. Sin mirar la solución, diseña la siguiente interfaz de usuario:



```

1. export default function App() {
2.   return (
3.     <View style={styles.contenedorPrincipal}>
4.       <View style={styles.fila}>
5.         <TextInput
6.           style={styles.cuadroTexto}
7.           placeholder={"Escribe un número positivo"}
8.           placeholderTextColor={"#666"}
9.         />
10.        <Pressable
11.          style={ ({pressed}) => pressed? [styles.boton,{opacity:0.4}]:styles.boton}
12.          onPress={aceptarPulsado}>
13.            <Text style={styles.textoBoton}>Aceptar</Text>
14.          </Pressable>
15.        </View>
16.      <View style={styles.contenedorSecundario}>
17.      </View>
18.    </View>
19.  )
20. }

```

6. Añade una variable de estado llamada **texto** con valor inicial una cadena de texto vacía, su setter **setTexto** y vincúlalo al **TextInput**

```

1. export default function App() {
2.   const [texto,setTexto]=useState("")
3.   return (
4.     <View style={styles.contenedorPrincipal}>
5.       <View style={styles.fila}>
6.         <TextInput
7.           style={styles.cuadroTexto}
8.           placeholder={"Escribe un número positivo"}
9.           placeholderTextColor={"#666"}
10.          value={texto}
11.          onChangeText={setTexto}
12.        />
13.      // resto omitido
14.    )
15.  }

```

7. Crea una carpeta llamada **utils** y en ella un archivo llamado **Funciones.ts**
8. En **Funciones.ts** crea y exporta una función llamada **toEnteroPositivo**, que reciba una cadena de texto y devuelva un objeto formado por estos campos:
- **valor**: es el resultado de convertir la cadena en entero positivo
  - **exito**: guardará **true** si se ha podido realizar la conversión

```

1. function toEnteroPositivo(cadenaTexto){
2.   const valor = parseInt(cadenaTexto)
3.   const éxito = !isNaN(resultado) && resultado > 0
4.   return {
5.     valor:valor,
6.     exito:exito
7.   }
8. }
9. export {toEnteroPositivo}

```

*La función **Number** convierte en número la cadena de texto que recibe, y devuelve **NaN** en caso de que dicha cadena no se pueda convertir en número.*

*La función **isNaN** devuelve **true** si el argumento que recibe es **NaN***

9. Simplifica la función anterior, poniendo directamente en el objeto las variables y usando sus nombres como claves.

```

1. function toEnteroPositivo(cadenaTexto){
2.     const valor = parseInt(cadenaTexto)
3.     const exito = !isNaN(resultado) && resultado > 0
4.     return { valor, exito }
5. }

```

En **JavaScript** podemos crear un objeto pasando directamente unas cuantas variables, y automáticamente se cogen como claves los nombres de las variables y como valores, los datos que contienen.

10. Voluntario: Vuelve a programar la función anterior, usando una **expresión regular** que compruebe si la cadena de texto tiene formato de número positivo

```

1. function toEnteroPositivo(cadenaTexto){
2.     const expresionRegular = /^\d+$/
3.     const exito = expresionRegular.test(cadenaTexto)
4.     const valor = parseInt(cadenaTexto)
5.     return { valor, exito }
6. }

```

Una **expresión regular** es un objeto que contiene un formato de texto (por ejemplo, el formato de un dni, el de una matrícula, etc) y su método **test** devuelve true si una cadena de texto se ajusta a ese formato.

En **JavaScript** se crean encerrando el patrón entre las barras / y /. En su interior aparecen símbolos que describen el patrón. Aquí hemos usado estos:

- **^** → Indica el inicio del patrón
- **\d** → Es un dígito entre 0 y 9
- **+** → Indica una o más apariciones del símbolo anterior (aquí es \d, o sea, un dígito entre 0 y 9)
- **\$** → Indica el final del patrón

Hay muchas más reglas, como estas:

- **Cualquier letra** → Indican la aparición de la correspondiente letra
- **\w** → Un carácter de la a-z (minúsculo o mayúsculo) y el guión bajo
- **.** → Cualquier carácter
- **\*** → Cero o más apariciones del símbolo anterior
- **?** → El símbolo anterior puede aparecer o no
- **{N}** → El símbolo anterior aparece exactamente N veces (N es un número)
- **{A:B}** → El símbolo anterior aparece de A a B veces (A y B son números)
- **{A:}** → El símbolo anterior aparece A o más veces (A es un número)
- **A-B** → Admite cualquier carácter entre A y B (A y B son caracteres)
- **[ ]** → Un carácter de entre todos los que hay entre los corchetes

11. En **App.tsx** crea una función llamada **aceptarPulsado** que compruebe si **texto** es un número entero positivo, y mostrará una ventana emergente si no es así.

```

1. function aceptarPulsado(){
2.     const conversion = toEnteroPositivo(texto)
3.     if(!conversion.exito){
4.         Alert.alert("Error","Debe introducirse un número entero positivo")
5.     }
6. }

```

12. Repite la programación de la función anterior, **desestructurando** el objeto **conversión** en dos variables que se llamen igual que sus claves

```
1. function aceptarPulsado(){
2.   const {éxito,valor} = toEnteroPositivo(texto)
3.   if(!éxito){
4.     Alert.alert("Error","Debe introducirse un número entero positivo")
5.   }
6. }
```

La **desestructuración** de un objeto permite guardar en variables los valores que contiene. En cada variable se guarda el valor del objeto cuya clave es el nombre de la variable. No es necesario pasar variables para todas las claves, sino solo para las que nos interesa recuperar, y además no importa el orden de la desestructuración.

Al desestructurar un objeto, evitamos tener que escribirlo cuando queremos acceder a sus campos (o sea, no escribimos **conversión.éxito** sino solo **éxito**)

13. Haz que al pulsar el botón de aceptar se llame a **aceptarPulsado**

```
1. <Pressable style={styles.boton} onPress={aceptarPulsado}>
2.   <Text style={styles.textoBoton}>Aceptar</Text>
3. </Pressable>
```

## 2.- Implementación del cálculo de divisores

Vamos a hacer que al pulsar el botón de aceptar se calculen todos los divisores del número introducido.

1. Añade una variable de entorno llamada **listaDivisores**, con su setter **setListaDivisores**. Será una lista vacía que se rellenará con los divisores del número que hemos introducido

```
1. export default function App() {
2.   const [listaDivisores, setListaDivisores] = useState([])
3.   // resto omitido
4. }
```

2. En el archivo **Funciones.ts** añade una función llamada **calcularDivisores**, que reciba un número y nos devuelva la lista de divisores de ese número

```
1. function calcularDivisores(numero){
2.   const lista=[]
3.   for(let i=1;i<=numero;i++){
4.     if(numero%i === 0){
5.       lista.push(i)
6.     }
7.   }
8.   return lista
9. }
```

Existen otras formas más eficientes de obtener la lista de divisores, pero por simplicidad vamos a mantener esta

3. Completa la función **aceptarPulsado** para que se calcule la lista de divisores en caso de que el texto introducido sea un número, y cuando la lista esté disponible se guarde en **listaDivisores**

```

1. function aceptarPulsado(){
2.   const {exito,valor} = toEnteroPositivo(texto)
3.   if(exito){
4.     const lista = calcularDivisores(valor)
5.     setListaDivisores(lista)
6.   }else{
7.     Alert.alert("Error","Debe introducirse un número entero positivo")
8.   }
9. }

```

### 3.- FlatList

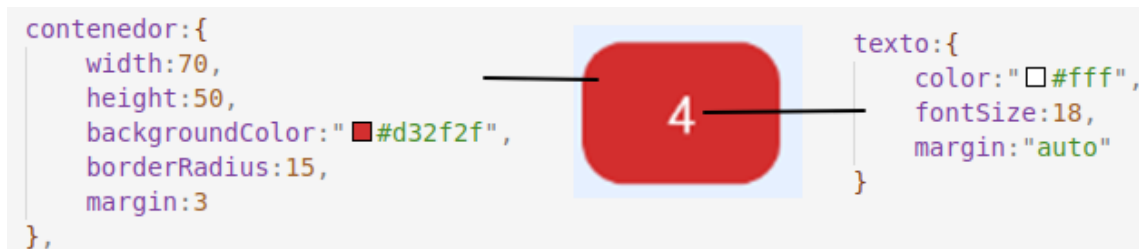
Vamos a mostrar en una lista los divisores obtenidos. En React Native el componente que permite mostrar una lista de datos se denomina **FlatList**. Es un componente muy usado, puesto que está diseñado para optimizar la memoria, mostrando solamente componentes para los datos que caben en la pantalla.

Los principales props del **FlatList** son:

- **data:** La lista de datos que deberá mostrarse en la lista. En nuestro caso, será la variable **listaDivisores**
- **renderItem:** Es el componente que se usará para mostrar cada dato individual de la lista. Para que funcione, dicho componente debe tener un prop llamado **item**. En nuestro caso, ese componente será un rectángulo que encerrará un número divisor.
- **keyExtractor:** Internamente el **FlatList** usa generación dinámica de componentes, y por ese motivo necesita asignar un prop **key** a los datos de la lista. En el prop **keyExtractor** pondremos una lambda que toma un dato de la lista (en nuestro caso un número) y devuelve un **string** con su clave primaria (en nuestro caso, servirá ese mismo número, pero pasado a **string**)

Comenzaremos diseñando el componente llamado **CajaDivisor**, que será el lugar donde se mostrará cada divisor. Es obligatorio que tenga un prop llamado **item**

1. Crea una carpeta llamada **components** y en ella el archivo **CajaDivisor.tsx**
2. En **CajaDivisor.tsx** teclea **rnfs + Intro**
3. Añade a la función **CajaDivisor** un prop llamado **item**
4. Sin mirar la solución, diseña **CajaDivisor** de esta forma:



```

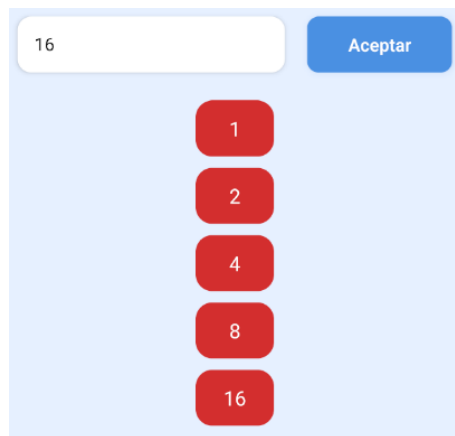
1. export default function CajaDivisor({item}) {
2.   return (
3.     <View style={styles.contenedor}>
4.       <Text style={styles.texto}>{item}</Text>
5.     </View>
6.   )
7. }

```

5. En **App.tsx** pon dentro del contenedor secundario una etiqueta **FlatList** con los siguientes valores para sus props:
  - **data** → La variable **listaDivisores**
  - **renderItem** → El componente **CajaDivisor**
  - **keyExtractor** → Una lambda que reciba un número y lo devuelva a si mismo, pero convertido en **string**

```
1. <View style={styles.contenedorSecundario}>
2.   <FlatList
3.     data={listaDivisores}
4.     renderItem={ CajaDivisor }
5.     keyExtractor={ numero => numero.toString()}
6.   />
7. </View>
```

6. Ejecuta la app y comprueba que al poner un número se muestra la lista de sus divisores



7. Prueba el número **222222** y comprueba que el **FlatList** lleva un **ScrollView** incorporado, de manera que puedes desplazarlo para ver todos los números.

#### 4.- Incorporar varias columnas al FlatList

Como los cuadros de los números son muy pequeños, podemos distribuirlos en columnas. Para ello, el **FlatList** cuenta con el prop **numColumns**, al que le pasaremos la cantidad de columnas que queramos utilizar

1. Añade al componente **FlatList** el prop **numColumns** y pásale el valor **5**

```
1. <View style={styles.contenedorSecundario}>
2.   <FlatList
3.     data={listaDivisores}
4.     renderItem={ CajaDivisor }
5.     keyExtractor={ numero => numero.toString()}
6.     numColumns={5}
7.   />
8. </View>
```

Al actualizar el archivo verás que el emulador da un error. Eso es normal, porque el **FlatList** no tiene aún implementado el cambio de columnas al vuelo durante el desarrollo. Deberás volver a abrir la app desde el menú de la terminal

2. Ejecuta la app y verás que ahora se distribuyen correctamente los datos en las columnas.

## 5.- Versión TypeScript

---

Como siempre, terminamos escribiendo la versión **TypeScript** de la app

1. Abre **Funciones.ts** y crea un tipo llamado **ResultadoConversion** que tenga dos campos:
  - **valor**, de tipo **number**
  - **exito**, de tipo **boolean**

```
1. type ResultadoConversion = {  
2.   valor:number  
3.   exito:boolean  
4. }
```

2. Añade los tipos de datos a las cabeceras de las funciones de **Funciones.ts**

```
1. function toEnteroPositivo(cadenaTexto:string):ResultadoConversion{  
2.   // resto omitido  
3. }  
4. function calcularDivisores(numero:number):Array<number>{  
5.   // resto omitido  
6. }
```

3. Abre **CajaDivisor.tsx** y crea el tipo **CajaDivisorProps**, con un campo llamado **item** de tipo **number**. Pon ese tipo al parámetro de la función **CajaDivisor**

```
1. type CajaDivisorProps = {  
2.   item:number  
3. }  
4. export default function CajaDivisor({item}:CajaDivisorProps) {  
5.   // resto omitido  
6. }
```

4. En **App.tsx** haz que la **useState** sepa que **listaDivisores** debe ser de tipo **Array<number>**

```
1. export default function App() {  
2.   const [listaDivisores, setListaDivisores] = useState<Array<number>>([])  
3.   // resto omitido  
4. }
```

5. Haz un commit titulado "finalizado tutorial 11"
6. Sitúate en la rama **main**
7. Mezcla en la rama **main** la rama **tutorial11**
8. Haz un **push**